Cleaning the AI Sec Watch Dataset

Olena McMurtrey

University of San Diego

AAI-501: Introduction to Artificial Intelligence

November 12, 2026

Summary

We cleaned the AI Sec Watch Dataset version 4.0 (Luu, 2026) into one comma-separated values file that is ready for analysis. Every problem we found was resolved in the data itself, not just flagged: Duplicates were removed, contradictory ratings and impossible dates were corrected, and the columns that would reveal the severity rating were removed. Each correction is recorded in a cleaning_note column so it can be traced.

Design

The raw data is never edited; cleaned files are produced from it by code (DrivenData, n.d.). The original Zenodo file is our master copy, and every row in the clean file keeps its id, so any original value can be looked up. The notebook rebuilds the clean file from the raw file in one run.

Results at a Glance

Table 1 summarizes the results of the cleaning. The raw file of 8,260 records in 48 columns became a clean file of 8,236 records in 41 columns, and the raw file itself was not changed. Every final check, including the check for Common Vulnerability Scoring System (CVSS) wording in the model text, returned zero problems.

Table 1

Cleaning Results at a Glance

Measure	Result
Raw data set (master, unchanged)	8,260 records, 48 columns
Clean data set	8,236 records, 41 columns, one comma-separated values file, ready for analysis
Record types	4,589 news, 2,876 vulnerability, 691 research, 71 regulatory, 9 incident
Problems resolved in the data	24 duplicates removed, 10 ratings corrected, 5 dates corrected
Columns removed	12 that leak the target, 3 that never change
Train and test	2,444 rated vulnerabilities: 1,955 train, 489 test
Prediction set (no official rating)	431 vulnerabilities (363 similar to training, 68 not)
Final checks	0 leaky columns, 0 CVSS wording in model text, 0 rating conflicts, 0 impossible dates, 0 repeated ids

Note. CVSS = Common Vulnerability Scoring System.

Problems Found and How They Were Resolved

Table 2 lists each problem found in the raw data and how it was resolved. Each correction is also recorded in the cleaning_note column of the row it changed, so it can be traced back to the raw file. Rating labels were checked against the CVSS rating bands given in the note under the table.

Table 2

Problems Found in the Raw Data and How They Were Resolved

Problem	How it was resolved
24 duplicate copies (14 news, 10 research)	Removed. The most complete record in each group was kept.
10 ratings that contradict their CVSS score	Corrected to the score's rating (8 low to medium, 2 medium to high). Each score was first recomputed from its CVSS 3.1 vector and matched.
5 publication dates after the collection date	Replaced by the collection time, the data set's own fallback rule.
14 unrated records with only an old CVSS 2.0 score	Left unrated in the prediction set. CVSS 2.0 has no critical band, so its scores cannot be converted to our four classes.
1 record rated "none" (CVSS 0.0)	Kept as "none," outside train and test, since the models predict four classes.
Severity wording in vulnerability text	Removed in new clean text columns; the original text is kept for reference.
Repeated items inside lists	Removed from 168 weakness lists and 5 vendor lists, where a repeat carries no new meaning. Package lists were left as the source gave them, because a repeated name there most likely means two different package registries.

Note. Rating bands follow the CVSS standard: 0.1 to 3.9 low, 4.0 to 6.9 medium, 7.0 to 8.9 high, and 9.0 to 10.0 critical (National Institute of Standards and Technology [NIST], n.d.). Scores were recomputed with the cvss package (Kontar & Red Hat Product Security, n.d.).

How the Cleaning Was Done

The notebook runs the cleaning in 10 steps, in the following order:

1. Loaded the raw file of 8,260 records in 48 columns.
2. Removed duplicates. A record is a copy when it has the same source and title as another, plus the same cleaned link or the same publication day. Same-title articles from different outlets were kept, since they are separate pieces.

3. Corrected labels and dates as shown in Table 2.
4. Removed leaky and constant columns (see Table 3).
5. Wrote cleaned text into new columns. In 513 vulnerability summaries and 34 titles, sentences about scoring (such as “rates it CVSS 5.3 (MEDIUM)”) were removed, and rating words that labeled the flaw were deleted. Words that describe the attack, such as “low-privileged user,” were kept.
6. Built the model text. Stock product descriptions were removed, because 443 TensorFlow advisories share one template sentence that a model learned as a sign of “low.” Common Vulnerabilities and Exposures (CVE) numbers and dates became neutral tokens, since they reveal the year.
7. Removed repeated values in list columns, with one exception. Some cells listed an item twice, for example “CWE-77; CWE-94; CWE-77.” In weakness lists, this happens because the National Vulnerability Database merges the weakness codes from the CVE’s publisher and from the National Institute of Standards and Technology’s analysts, so a repeat means both named the same weakness; it is counted once. Package lists were not changed: The data set drops the package registry, so a repeated name such as deepseek-tui most likely means two packages, one on npm and one in Rust’s registry.
8. Kept the meaning of missing values. An empty patch field means the source did not say, so it stays empty; a separate column marks the 1,562 records that report a fix. Empty cells elsewhere mostly mean “not applicable”; for example, a news article has no CVE number.
9. Set the target and the time-based split. Rated vulnerabilities published before July 30, 2026, form the training set (80%), and later ones form the test set (20%).

10. Checked and exported. The automated checks returned zero leaky columns, zero CVSS wording in model text, zero rating conflicts, zero impossible dates, and zero repeated ids.

Columns Removed

Table 3 lists the 15 columns removed from the clean file. Twelve columns would leak the target because they state or reveal the severity rating, and three hold the same value in every record. Removing them leaves 33 of the 48 raw columns.

Table 3

Columns Removed From the Clean File and Why

Columns removed	Reason
cvss_score, cvss_vector	State the rating directly
attack_vector, attack_complexity, privileges_required, user_interaction	Parts of the CVSS score, read from the vector
severity	Copies the official rating for rated records
severity_source	Shows whether an official rating exists
epss_score, epss_checked_at, exploit_maturity, kev_date_added	Exploit data read on October 10, 2026, after most records were published, and linked to severity
classifier_model, classifier_prompt_version, summary_prompt_version	Same value in every record

Columns Added or Renamed

Table 4 lists the columns added to the clean file and the one column renamed. The renamed column, severity_label, replaces cvss_severity and holds the corrected rating. Together with the 33 kept raw columns, the eight new columns bring the clean file to 41 columns.

Table 4

Columns Added or Renamed in the Clean File

Column	What it holds
severity_label	The corrected official rating (replaces cvss_severity): low, medium, high, critical, or none
split	train or test (by date) for rated vulnerabilities; predict for unrated ones; empty for other records
title_clean, summary_clean	Tidied text; severity wording removed from vulnerability records
model_text	The text the models read (vulnerabilities): clean title and summary, product boilerplate removed, CVE numbers and dates replaced by neutral tokens
cleaning_note	What was corrected in the row and why (30 rows)
similar_to_training	Prediction rows: 1 if the source and era match the training data
is_tensorflow	1 for TensorFlow advisories, to compare results with and without them
patch_status_known	1 where the source reported a fix

Class Balance for Training and Testing

Table 5 shows how the 2,444 rated vulnerabilities fall into the four severity classes in the training and test sets. High is the largest class in both sets, and low is the smallest. The test set holds a larger share of high ratings than the training set (48.9% vs. 40.4%).

Table 5

Severity Classes in the Training and Test Sets

Severity	Train, *n* (%)	Test, *n* (%)	Total, *n*
Low	156 (8.0)	19 (3.9)	175
Medium	721 (36.9)	173 (35.4)	894
High	789 (40.4)	239 (48.9)	1,028
Critical	289 (14.8)	58 (11.9)	347
Total	1,955 (100.0)	489 (100.0)	2,444

Note. Values are counts, with the share of each set in parentheses. Low and critical are the rare classes, so models should use class weighting and be judged on macro F1 and on recall for the critical class.

Rules for Analysis and Modeling

All analysis and modeling should follow four rules:

- Rows: Train on split = train, evaluate on split = test, and rate split = predict.
- Model inputs: model_text, cwe_ids, attack_type, impact_type, affected_vendors, ai_component_targeted, attack_sophistication, llm_specific, patch_status_known, and source_name, and nothing else.
- Never model inputs: title and summary (they still hold severity wording), severity_label (the target), split, and cleaning_note.
- Dates: published_at keeps its original ISO 8601 form; parse it as Coordinated Universal Time.

Limitations

The clean data set has four limitations:

- Ratings mix CVSS versions 3.1 and 4.0, which share rating bands but score differently, so some label noise remains.
- Attack type, impact, and similar fields were assigned by an AI classifier and may contain errors.
- The text scrub follows written rules, checked by automated tests and sample reading; rare phrasings may remain.
- Only 19 low-rated records fall in the test set, so recall for that class will vary between runs.

References

DrivenData. (n.d.). *Opinions*. Cookiecutter Data Science. https://cookiecutter-data-science.drivendata.org/opinions/

Luu, T. (2026). *AI Sec Watch dataset: AI/LLM security threat intelligence* (Version 4.0) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.19519469

National Institute of Standards and Technology. (n.d.). *Vulnerability metrics*. National Vulnerability Database. https://nvd.nist.gov/cvss.cfm